

Object–Oriented Programming

Lab 12: Exception Handling

ENSIA 2024-2025

Exercise 1

Different exceptions can arise when doing mathematical calculations. We want you to define various classes to capture and handle such exceptions.

1. Define the class `MathException`, derived from the class `exception`, and which has a protected string data member `message`, a constructor, and a function `what()` which returns the value of `message`.
2. Define three classes:
 - (a) `DivideByZeroException`
 - (b) `OverflowException`
 - (c) `RootOfNegativeException`

Each of these classes is derived from the class `MathException` and has a constructor which passes the appropriate message to the `MathException` constructor.

- An object of the first class is thrown when a division by zero is about to occur on two integer values entered by the user.
 - An object of the second class is thrown when (just for the purposes of this exercise) a very big number (say $> 5\,000\,000$) is read.
 - An object of the third class is thrown when the calculation of the root of a negative number entered by the user is about to be calculated.
 - Otherwise, the operations are performed normally. If no `OverflowException` occurs, simply output the number that has been read.
3. Create a diagram illustrating the hierarchical relationship of the various exception classes in terms of inheritance.
 4. Write a `main` program to try the three above exceptions: division, overflow, and square root. The main includes a `try` block. Within the `try` block, handle the necessary actions to throw the appropriate exception each time. Following the `try` block, include `catch` blocks that match the exceptions and handle them by displaying the appropriate messages.
 5. Modify the `main` program to handle all the math exceptions using only the `MathException` class for exception handling.
 6. What is your conclusion if you are asked to answer this question: should you use one `try` block and multiple `catch` blocks, or multiple `try` and multiple `catch` blocks?

Note: You can use one file to implement the code for this exercise.

Exercise 2

```
#include <iostream>
using namespace std;

int main() {

    int num = 0;

    do {
        cout << "Enter a number between 1 and 10" << endl;
        cin >> num;

        if (num < 1 || num > 10) {
            cout << "Illegal value " << num << " entered. Try again." << endl;
        }

    } while (num < 1 || num > 10);

    cout << "Value " << num << " correctly entered! Thank you." << endl;
}
```

Questions:

- Write the program above and compile it. Run and enter an integer between 1 and 10, then enter "four" in letters. What happens?
- By default, C++ does not throw exceptions when something goes wrong with IO streams. Activate exception-throwing by prepending this line of code at the start of the `main` method:

```
cin.exceptions(iostream::failbit);
```

- To handle the exception, add a try/catch block to catch and handle the `iostream::failure` exception. Use the catch block given below. You will also need to include the `<limits>` header.

```
#include <limits>

catch (iostream::failure& iof) {
    cout << "Non-integer value. Please enter a number." << endl;
    cin.clear(); // reset error flags
    cin.ignore(numeric_limits<streamsize>::max(), '\n'); // clear
buffer
}
```

- Recall that the program is requesting a number between 1 and 10. Run the program again and enter 10.8. Although this number is greater than 10, the program will think you entered simply 10 and tell you it is a valid value.
- C++ does not offer an exception for this situation, so we will have to make our own. Include this code in the program after you write the user's input to `num`:

```
// if this number was part of a non-integer entry, throw an exception
if (cin.peek() != '\n') {
    throw &num;
}
```

```

catch (int* pnum) {
    cout << "Non-integer value. You entered something after " << *pnum;
    cin.clear(); // reset error flags
    cin.ignore(numeric_limits<streamsize>::max(), '\n'); // clear
buffer
    *pnum = 0; // reset num to its initial value
}

```

- Compile and run the program again. Test it with a variety of input (integers, floats, characters, etc.). The program should not abort when floats or character data are given.

Exercise 3

In the problem of Exercise 3 of Lab 11 (implementation of the `DoubleSubscriptedArray` class), if the indices of a desired entry in the two-dimensional array were invalid (for example, out of range), you were asked to print an error message and exit the program.

Now, modify your program to instead throw an `ArrayOutOfRangeException` exception if either the row or column indices are invalid. You should define the `ArrayOutOfRangeException` exception class.